MODULE 3 · CONFIGURATION

ConfigMaps

Decouple configuration from your container images

WHY IT MATTERS

Config baked into an image means a rebuild per environment

Hardcode a database host or log level into your image and every environment change means a new image, a new tag, a new rollout. That doesn't scale past one environment.

A **ConfigMap** holds that configuration as its own Kubernetes object, separate from the image. The same image runs in dev, test, and prod — only the ConfigMap it's paired with changes.

BY THE END YOU CAN

- Create a ConfigMap from literals and from a file
- Inject its data as environment variables
- Mount it as files in a volume
- Know which method needs a Pod restart to see changes

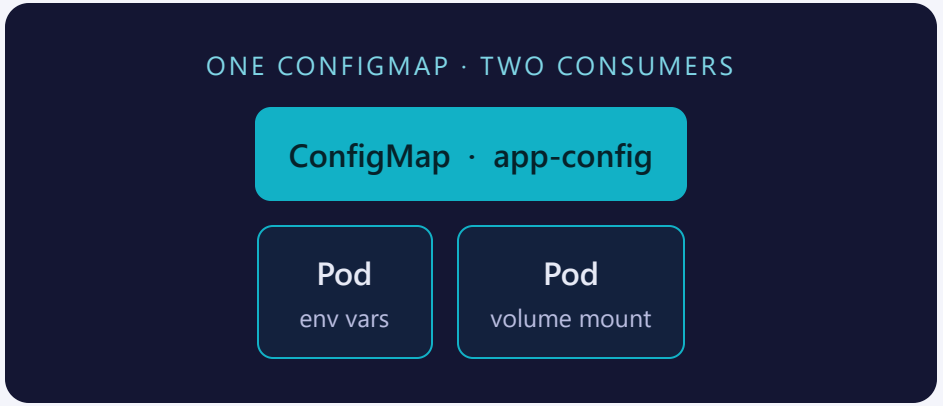
CONCEPT

A ConfigMap is just key-value data, stored by the cluster

A ConfigMap is a plain object holding non-sensitive configuration as key-value pairs. It has no `spec`, no lifecycle of its own — it's just data that Pods can reference.

- Created from literal `key=value` pairs
- Created from the contents of a file
- Referenced by name from a Pod — not copied in at build time

Namespaced like Pods — a ConfigMap in `training` is invisible to Pods in another namespace.



CREATING A CONFIGMAP

From literal values, or from a file

FROM LITERAL VALUES	FROM A FILE
`--from-literal=KEY=VALUE` or a `data:` map	`--from-file=path/to/file` or a `data:` block
Good for a handful of short settings	Good for whole config files (`.properties` , `.conf` , `.json`)
Each flag becomes one key	The **filename** becomes the key; the whole file is the value

MENTAL MODEL

Same object either way — `ConfigMap.data` is always just a map of strings. Only how you fill it differs.

MANIFEST ANATOMY

No spec — just data

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-literal   # how Pods reference it
  namespace: training
data:
  APP_ENV: training
  LOG_LEVEL: debug
```

- **apiVersion / kind** — `v1` / `ConfigMap`, no `spec` block
- **metadata.name** — the handle a Pod's env or volume config points at
- **metadata.namespace** — must match the Pod's namespace to be visible to it
- **data** — flat map of string `key: value` pairs
- A file-based ConfigMap stores the whole file's contents under one key

File-based manifest (`configmap-file.yaml`) is in the lab.

DO IT — TWO WAYS

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF	DECLARATIVE — VERSIONABLE, REPEATABLE
`kubectl create configmap app-config --from-literal=APP_ENV=training`	`kubectl apply -f configmap-literal.yaml`
Good for scratch testing	You own the file; it's reviewable and re-appliable

THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl create configmap app-config \
    --from-literal=APP_ENV=training \
    --dry-run=client -o yaml > configmap.yaml
```

SEE IT — ENVIRONMENT VARIABLES

envFrom pulls every ConfigMap key in as an env var

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-configmap-env
  namespace: training
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - echo APP_ENV=$APP_ENV LOG_LEVEL=$LOG_LEVEL && sleep 3600
      envFrom:
        - configMapRef:
            name: app-config-literal
  restartPolicy: Never
```

`pod-env.yaml` — every key in `app-config-literal` lands as an env var in the container.

SEE IT — VOLUME MOUNT

The ConfigMap becomes a file the container reads

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-configmap-volume
  namespace: training
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - cat /etc/config/app-config.properties && sleep 3600
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config-file
  restartPolicy: Never
```

pod-volume.yaml — verify both Pods: kubectl exec pod-configmap-env -n training -- env | grep -E "APP_ENV|LOG_LEVEL" and kubectl exec pod-configmap-volume -n training -- cat /etc/config/app-config.properties.

ENV VARS VS MOUNTED VOLUME

Only one of these updates itself

ENVIRONMENT VARIABLES	MOUNTED VOLUME
Set once, at container start	Kubelet re-syncs the mounted files on a periodic interval
Editing the ConfigMap has zero effect on a running Pod	Editing the ConfigMap updates the file in place, no restart
A Pod **restart** is required to pick up new values	Your app still has to notice the file changed and re-read it

THE GOTCHA

An app that only reads env vars at startup will never see a ConfigMap update — even minutes later — until its Pod is restarted.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Expecting env vars to update live.** They're baked in at container start — nothing short of a restart changes them.
- **Mounting a single key with `subPath`.** That breaks the automatic refresh; the path is never updated.
- **ConfigMap missing when the Pod starts.** A Pod referencing one that doesn't exist yet won't start.
- **Storing secrets in a ConfigMap.** Data is plain text, visible to anyone who can `get` it — that's what Secrets (Lab 3.2) are for.

YOUR CONFIGMAP TOOLKIT

Commands to keep at hand

```
$ kubectl create configmap NAME --from-literal=K=V   # from literals
$ kubectl create configmap NAME --from-file=FILE      # from a file
$ kubectl get configmaps -n NS                        # list
$ kubectl describe configmap NAME -n NS               # see keys + values
$ kubectl exec POD -- env | grep KEY                  # confirm env var
$ kubectl exec POD -- cat /etc/config/FILE            # confirm mounted file
```

Tip: `kubectl edit configmap NAME`, then re-check the mounted file — no restart needed to see it move.

RECAP

ConfigMaps in one breath

- ConfigMap = key-value config, created from literals or a file
- Consumed as environment variables or as a mounted volume
- Env vars freeze at container start; volumes refresh automatically
- Non-sensitive settings only — Secrets are next

HANDS-ON

Now run `Lab 3.1`

`labs/3.1/`

Next: **3.2 — Secrets**